

Übersetzung und Analyse von Programmen

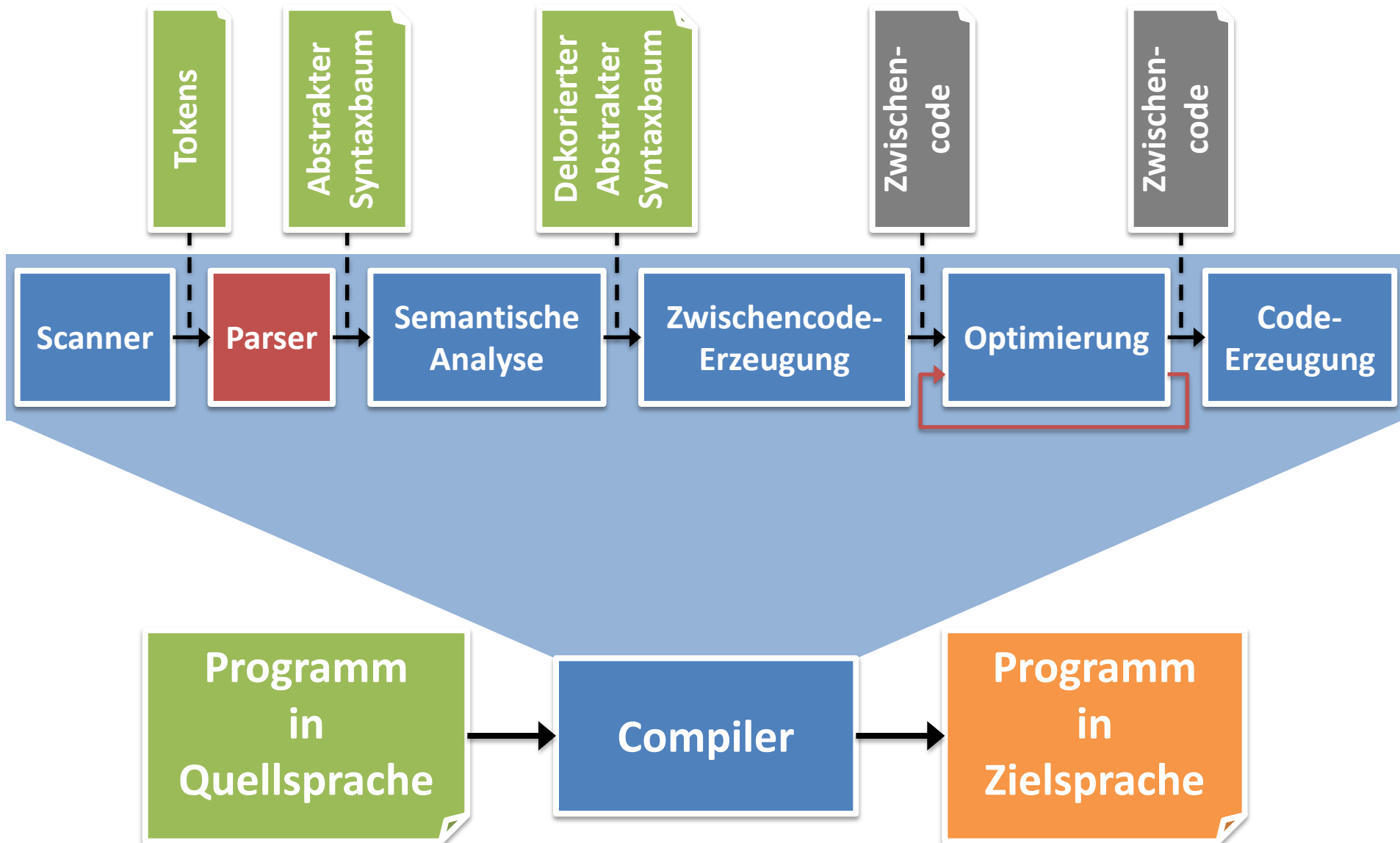
A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green geometric shapes.

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl

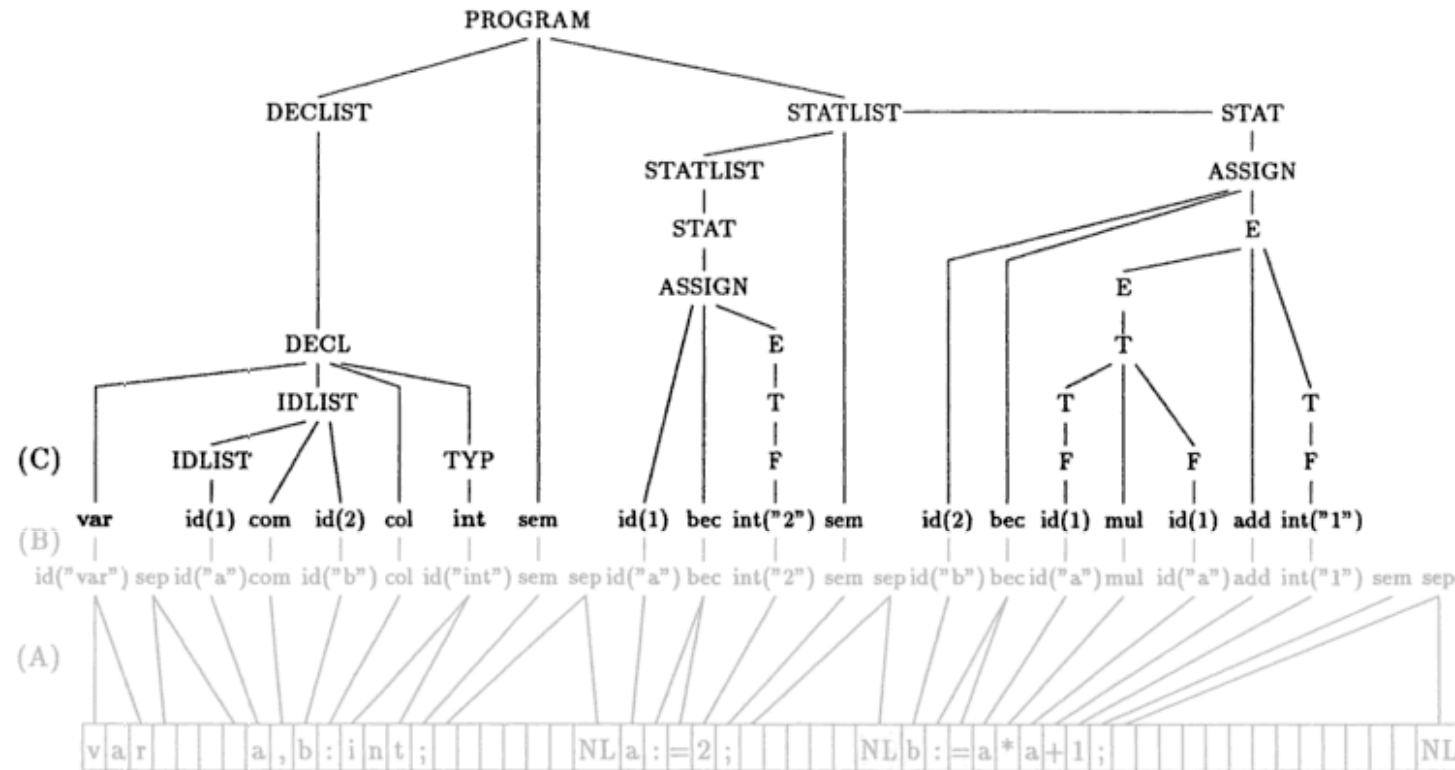
Informatik, Universität Trier

Struktur eines Compilers



Struktur eines Compilers

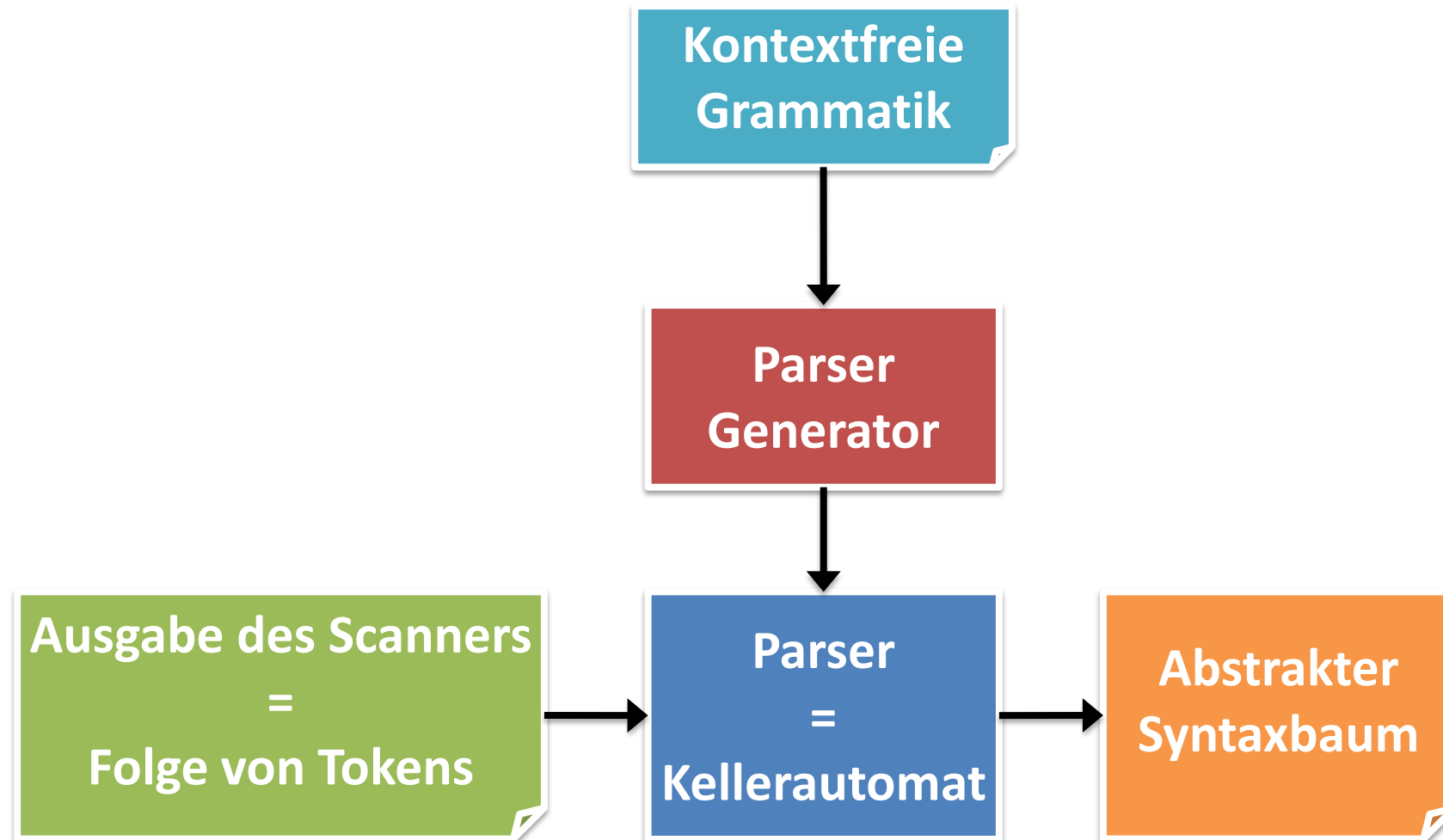
Parser



Quelle: Übersetzerbau, Wilhelm & Maurer, 1992

- (A) Lexikalische Analyse
- (B) Filtern/Sieben
- (C) Syntaktische Analyse

Syntaktische Analyse

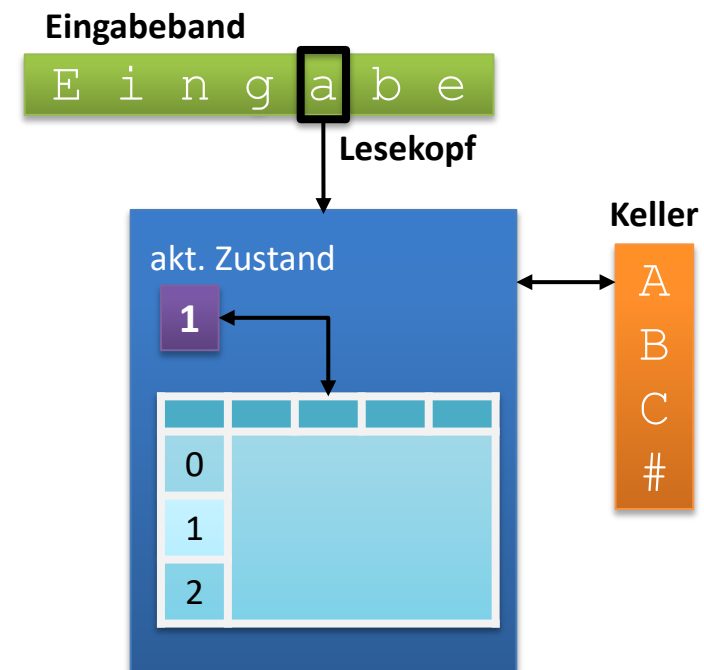


Kellerautomaten

Ein **nichtdeterministischer** Kellerautomat ist ein 7-Tupel $M = (Z, \Sigma, \Gamma, \delta, z_0, \#, F)$, wobei

- Z eine endliche Menge von **Zuständen**,
- Σ das **Eingabealphabet**,
- Γ das **Kelleralphabet**,
- $\delta : Z \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}(Z \times \Gamma^*)$ die **Übergangsfunktion**, die nur auf *endliche* Teilmengen von $Z \times \Gamma^*$ abbildet,
- $z_0 \in Z$ der **Startzustand**,
- $\# \in \Gamma$ das **Anfangssymbol im Keller** und
- $F \subseteq Z$ eine Menge von **finalen Zuständen**

ist.



Kontextfreie Grammatiken

Eine kontextfreie Grammatik ist ein Quadrupel $G = (S, N, T, P)$, wobei N und T disjunkt und

- $S \in N$ das **Startsymbol**,
- N eine endliche Menge von **Nichtterminalsymbolen**
- T eine endliche Menge von **Terminalsymbolen**
- $P \subseteq N \times (N \cup T)^*$ eine endliche Menge von **Produktionen** (Ableitungsregeln)

Ein Wort $w \in (N \cup T)^*$ heißt **Satzform**.

Ein Beispiel: $x - 2 * y$

Grammatik

1	<i>Goal</i>	$\rightarrow$	<i>Expr</i>
2	<i>Expr</i>	$\rightarrow$	<i>Expr Op Expr</i>
3			<u>number</u>
4	Kopf		<u>id</u>
5	<i>Op</i>	$\rightarrow$	+
6			-
7			*
8			/

Note: In the original image, an orange box highlights 'Expr' in row 2, and a blue box highlights 'Expr Op Expr' in row 2. An orange arrow points from 'Kopf' to the first 'Expr' in row 2. A blue arrow points from 'Rumpf' to the 'Op' in row 2.

$S = Goal$

$N = \{Goal, Expr, Op\}$

$T = \{\underline{number}, \underline{id}, +, -, *, /\}$

Ableitung

$G \Rightarrow$

Kurz: $Expr \Rightarrow^* \underline{id} - \underline{num} * \underline{id}$

Die Suche nach einer Ableitung wird als **Parsing** bezeichnet.

Ein Beispiel: $x - 2 * y$

Grammatik

1	<i>Goal</i>	→	<i>Expr</i>
2	<i>Expr</i>	→	<i>Expr Op Expr</i>
3			<u>number</u>
4	Kopf		<u>id</u>
5	<i>Op</i>	→	+
6			-
7			*
8			/

Rumpf

$S = Goal$

$N = \{Goal, Expr, Op\}$

$T = \{\underline{number}, \underline{id}, +, -, *, /\}$

Ableitung

Regel	Satzform
—	<i>Expr</i>
2	<i>Expr Op Expr</i>
4	$\langle id, \underline{x} \rangle Op Expr$
6	$\langle id, \underline{x} \rangle - Expr$
2	$\langle id, \underline{x} \rangle - Expr Op Expr$
3	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle Op Expr$
7	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle * Expr$
4	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle * \langle id, \underline{y} \rangle$

Kurz: $Expr \Rightarrow^* \underline{id} - \underline{num} * \underline{id}$

Die Suche nach einer Ableitung wird als **Parsing** bezeichnet.

Ableitungen

- In jedem Schritt wird ein Nichtterminal ausgewählt und durch die rechte Seite einer Produktion ersetzt:

$$\alpha X \beta \Rightarrow \alpha \gamma \beta \quad \text{falls } (X \rightarrow \gamma) \in P$$

- Null bzw. einer oder mehrere Schritte: $\alpha \Rightarrow^* \beta$ bzw. $\alpha \Rightarrow^+ \beta$
- Verschiedene Auswahl kann zu verschiedenen Ableitungen führen.

Zwei Arten von Ableitungen im Folgenden wichtig:

- **Linksableitung**
 - das Nichtterminal, das **am weitesten links** steht, wird ersetzt ($\alpha \Rightarrow_{lm} \beta$).
- **Rechtsableitung**
 - das Nichtterminal, das **am weitesten rechts** steht, wird ersetzt ($\alpha \Rightarrow_{rm} \beta$).

lm = leftmost, rm = rightmost

1	<i>Goal</i>	$\rightarrow$	<i>Expr</i>
2	<i>Expr</i>	$\rightarrow$	<i>Expr Op Expr</i>
3			<u>number</u>
4			<u>id</u>
5	<i>Op</i>	$\rightarrow$	+
6			-
7			*
8			/

Ein Beispiel: **x - 2 * y**

Linksableitung

Regel	Satzform
—	<i>Expr</i>
2	<i>Expr Op Expr</i>
4	<i><id, <u>x</u>> Op Expr</i>
6	<i><id, <u>x</u>> - Expr</i>
2	<i><id, <u>x</u>> - Expr Op Expr</i>
3	<i><id, <u>x</u>> - <num, <u>2</u>> Op Expr</i>
7	<i><id, <u>x</u>> - <num, <u>2</u>> * Expr</i>
4	<i><id, <u>x</u>> - <num, <u>2</u>> * <id, <u>y</u>></i>

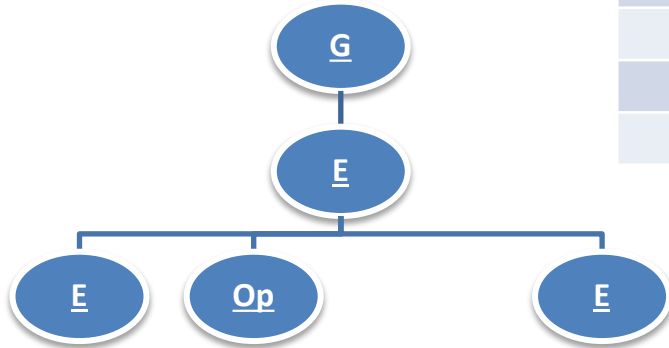
Rechtsableitung

Regel	Satzform
—	<i>Expr</i>
2	<i>Expr Op Expr</i>
4	<i>Expr Op <id, <u>y</u>></i>
7	<i>Expr * <id, <u>y</u>></i>
2	<i>Expr Op Expr * <id, <u>y</u>></i>
3	<i>Expr Op <num, <u>2</u>> * <id, <u>y</u>></i>
6	<i>Expr - <num, <u>2</u>> * <id, <u>y</u>></i>
4	<i><id, <u>x</u>> - <num, <u>2</u>> * <id, <u>y</u>></i>

Ableitungen und Syntaxbäume

Links-
ableitung

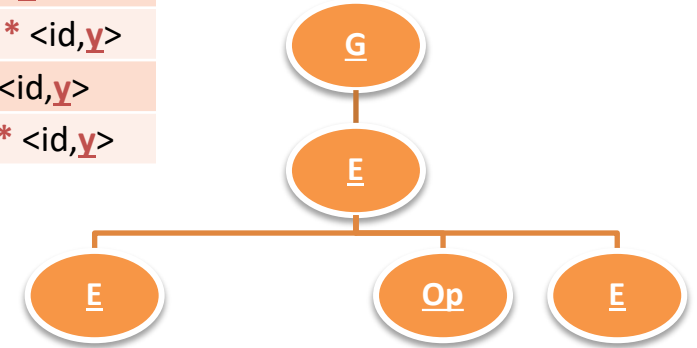
Regel	Satzform
—	$Expr$
2	$Expr \ Op \ Expr$
4	$\langle id, \underline{x} \rangle \ Op \ Expr$
6	$\langle id, \underline{x} \rangle \ - \ Expr$
2	$\langle id, \underline{x} \rangle \ - \ Expr \ Op \ Expr$
3	$\langle id, \underline{x} \rangle \ - \ \langle num, \underline{2} \rangle \ Op \ Expr$
7	$\langle id, \underline{x} \rangle \ - \ \langle num, \underline{2} \rangle \ * \ Expr$
4	$\langle id, \underline{x} \rangle \ - \ \langle num, \underline{2} \rangle \ * \ \langle id, \underline{y} \rangle$



$x-(2*y)$

Rechts-
ableitung

Regel	Satzform
—	$Expr$
2	$Expr \ Op \ Expr$
4	$Expr \ Op \ \langle id, \underline{y} \rangle$
7	$Expr \ * \ \langle id, \underline{y} \rangle$
2	$Expr \ Op \ Expr \ * \ \langle id, \underline{y} \rangle$
3	$Expr \ Op \ \langle num, \underline{2} \rangle \ * \ \langle id, \underline{y} \rangle$
6	$Expr \ - \ \langle num, \underline{2} \rangle \ * \ \langle id, \underline{y} \rangle$
4	$\langle id, \underline{x} \rangle \ - \ \langle num, \underline{2} \rangle \ * \ \langle id, \underline{y} \rangle$



$(x-2)*y$

Ableitungen und Syntaxbäume

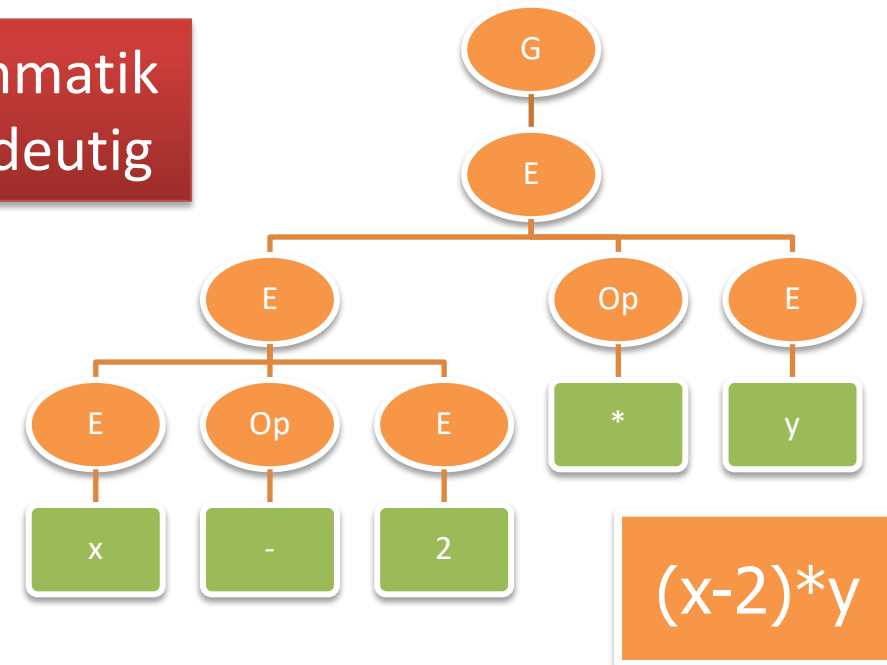
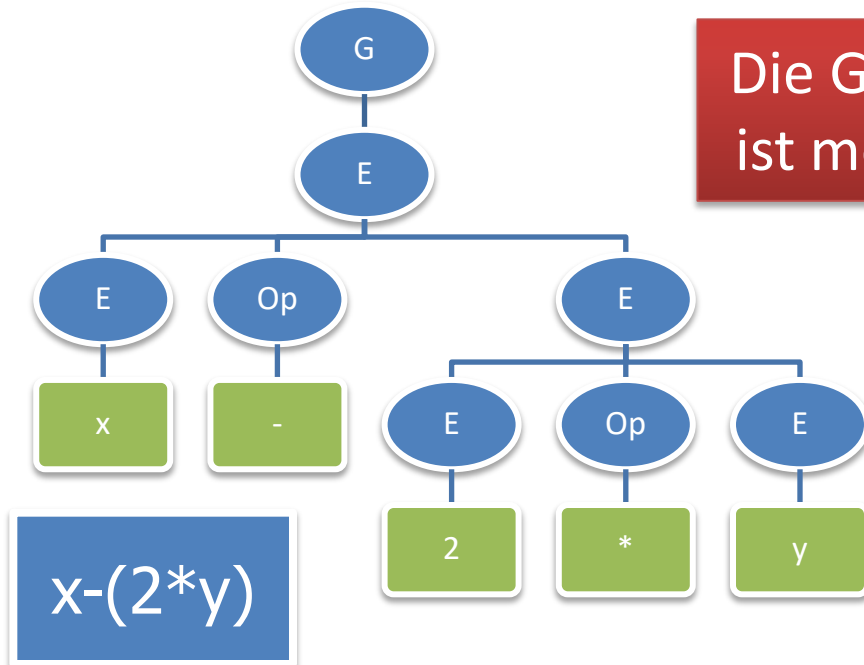
Links-
ableitung

Regel	Satzform
—	$Expr$
2	$Expr Op Expr$
4	$\langle id, \underline{x} \rangle Op Expr$
6	$\langle id, \underline{x} \rangle - Expr$
2	$\langle id, \underline{x} \rangle - Expr Op Expr$
3	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle Op Expr$
7	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle * Expr$
4	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle * \langle id, \underline{y} \rangle$

Regel	Satzform
—	$Expr$
2	$Expr Op Expr$
4	$Expr Op \langle id, \underline{y} \rangle$
7	$Expr * \langle id, \underline{y} \rangle$
2	$Expr Op Expr * \langle id, \underline{y} \rangle$
3	$Expr Op \langle num, \underline{2} \rangle * \langle id, \underline{y} \rangle$
6	$Expr - \langle num, \underline{2} \rangle * \langle id, \underline{y} \rangle$
4	$\langle id, \underline{x} \rangle - \langle num, \underline{2} \rangle * \langle id, \underline{y} \rangle$

Rechts-
ableitung

Die Grammatik
ist mehrdeutig



Mehrdeutigkeit

Eine Grammatik ist **mehrdeutig**, wenn es ein Wort gibt, für das

- zwei verschiedene *Rechtsableitungen* oder
- zwei verschiedene *Linksableitungen* oder
- zwei verschiedene *Syntaxbäume*

existieren.

Ein Klassiker unter den Mehrdeutigkeiten ...

Das *if-then-else* Problem (aka „dangling else“-Problem):

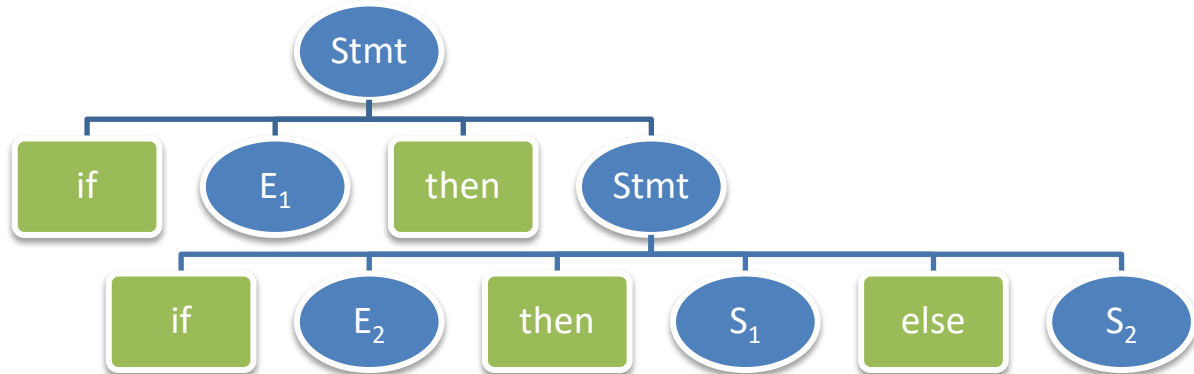
1	$Stmt \rightarrow \text{if } Expr \text{ then } Stmt$
2	$ \text{if } Expr \text{ then } Stmt \text{ else } Stmt$
3	$ \dots \text{ weitere Anweisungen } \dots$

Stmt $\Rightarrow$

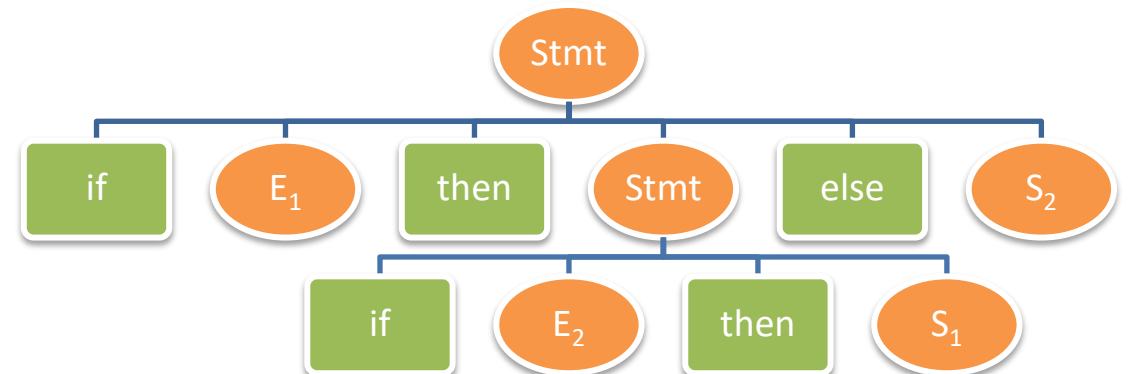
Stmt $\Rightarrow$

Die Satzform **if Expr₁ then if Expr₂ then Stmt₁ else Stmt₂** hat zwei Syntaxbäume:

a) Zuerst Produktion 1, dann Produktion 2



b) Zuerst Produktion 2, dann Produktion 1



a) ist Konvention in Java, C++, u.a.: ein else-Zweig bezieht sich immer auf die letzte if-Abfrage.

Mehrdeutigkeit in Programmiersprachen

Ursachen für Mehrdeutigkeiten in Programmiersprachen

- Unklarheiten in der kontextfreien Syntax (if-then-else)
- Unklarheiten, die nur mit Kontextwissen zu klären sind (Überladung)

Auflösen der Mehrdeutigkeit

- Im ersten Fall: Umschreiben der Grammatik
- Im zweiten Fall: Kooperation mit anderen Analysen
 - Information über Deklarationen, Typen, ...
 - Akzeptiere Obermenge von $L(G)$ und prüfe mit anderen Mitteln

Präzedenz

Die folgende Beispielgrammatik ist **eindeutig** und legt für Ausdrücke die Präzedenz, also eine Auswertungsreihenfolge, fest.

1	<i>Goal</i>	$\rightarrow$	<i>Expr</i>
2	<i>Expr</i>	$\rightarrow$	<i>Expr</i> + <i>Term</i>
3			<i>Expr</i> - <i>Term</i>
4			<i>Term</i>
5	<i>Term</i>	$\rightarrow$	<i>Term</i> * <i>Factor</i>
6			<i>Term</i> / <i>Factor</i>
7			<i>Factor</i>
8	<i>Factor</i>	$\rightarrow$	<u>number</u>
9			<u>id</u>

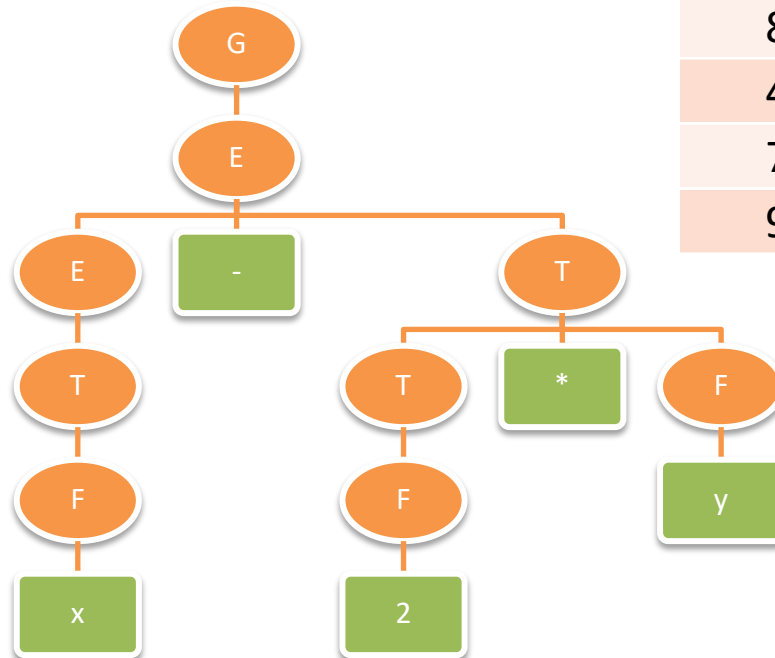
Verwendet Nichtterminal für jede Präzedenzebene

Punkt – vor Strichrechnung

- Höhere Präzedenz
- Niedriger Präzedenz

Rechtsableitung von **$x - (2 * y)$**

Regel	Satzform
–	<i>Expr</i>
3	<i>Expr</i> - <i>Term</i>
5	<i>Expr</i> - <i>Term</i> * <i>Factor</i>
9	<i>Expr</i> - <i>Term</i> * <id, <u>y</u> >
7	<i>Expr</i> - <i>Factor</i> * <id, <u>y</u> >
8	<i>Expr</i> - <num, <u>2</u> > * <id, <u>y</u> >
4	<i>Term</i> - <num, <u>2</u> > * <id, <u>y</u> >
7	<i>Factor</i> - <num, <u>2</u> > * <id, <u>y</u> >
9	<id, <u>x</u> > - <num, <u>2</u> > * <id, <u>y</u> >



Linksrekursion

Eine Grammatik ist **linksrekursiv**, wenn es ein Nichtterminal $A \in N$ und eine Ableitung $A \Rightarrow^+ A\alpha$ mit $\alpha \in (NT \cup T)^*$ gibt.

- Die Beispielgrammatik ist **direkt linksrekursiv**.
- Linksrekursion kann zum Nichtterminieren eines Top-down Parsers führen
- Kann man die Beispielgrammatik in eine Grammatik ohne Linksrekursion umwandeln?

1	<i>Goal</i>	$\rightarrow$	<i>Expr</i>
2	<i>Expr</i>	$\rightarrow$	<i>Expr</i> + <i>Term</i>
3			<i>Expr</i> - <i>Term</i>
4			<i>Term</i>
5	<i>Term</i>	$\rightarrow$	<i>Term</i> * <i>Factor</i>
6			<i>Term</i> / <i>Factor</i>
7			<i>Factor</i>
8	<i>Factor</i>	$\rightarrow$	<u>number</u>
9			<u>id</u>

Entfernen von Linksrekursion

Idee: Sei folgende Grammatik gegeben:

1	$Links \rightarrow Links \alpha$	$L \Rightarrow$
2	$\quad \quad \quad \beta$	

wobei weder α noch β mit *Links* beginnen und $\alpha \neq \varepsilon \wedge \beta \neq \varepsilon$.

Wir können diese Grammatik in eine nicht-linksrekursive umwandeln:

1	$Links \rightarrow \beta Rechts$	$L \Rightarrow$
2	$Rechts \rightarrow \alpha Rechts$	
3	$\quad \quad \quad \varepsilon$	

wobei *Rechts* ein neues Nichtterminal ist.

Hieraus lässt sich ein Algorithmus entwickeln, der beliebige **direkt linksrekursive** KFG ohne Zyklen und Epsilon-Produktionen in rechtsrekursive Grammatiken umwandelt.

Entfernen von Linksrekursion

- Einfaches Vorgehen:
 - Gruppieren der betroffenen Produktionen, so dass kein β_i mit *Links* beginnt:

Links $\rightarrow$ *Links* α_1 | ... | *Links* α_m | β_1 | ... | β_n

- Ersetzen der Produktionen durch:

Links $\rightarrow$ β_1 *Rechts* | ... | β_n *Rechts*

Rechts $\rightarrow$ α_1 *Rechts* | ... | α_m *Rechts* | ϵ

Entfernen von Linksrekursion

$$\text{Links} \rightarrow \text{Links } \alpha_1 \mid \dots \mid \text{Links } \alpha_m \mid \beta_1 \mid \dots \mid \beta_n$$

Die linksrekursive Ausdrucksgrammatik

$$\begin{array}{l} \text{Expr} \rightarrow \text{Expr} + \text{Term} \\ \quad \mid \text{Expr} - \text{Term} \\ \quad \mid \text{Term} \end{array}$$

$$\begin{array}{l} \text{Term} \rightarrow \text{Term} * \text{Factor} \\ \quad \mid \text{Term} / \text{Factor} \\ \quad \mid \text{Factor} \end{array}$$

Die umgewandelte Grammatik

$$\begin{array}{l} \text{Links} \rightarrow \beta_1 \text{Rechts} \mid \dots \mid \beta_n \text{Rechts} \\ \text{Rechts} \rightarrow \alpha_1 \text{Rechts} \mid \dots \mid \alpha_m \text{Rechts} \mid \varepsilon \end{array}$$

Entfernen von Linksrekursion

$$\text{Links} \rightarrow \text{Links } \alpha_1 \mid \dots \mid \text{Links } \alpha_m \mid \beta_1 \mid \dots \mid \beta_n$$

Die linksrekursive Ausdrucksgrammatik

$$\begin{aligned} \text{Expr} &\rightarrow \text{Expr} + \text{Term} \\ &\mid \text{Expr} - \text{Term} \\ &\mid \text{Term} \end{aligned}$$

$$\begin{aligned} \text{Term} &\rightarrow \text{Term} * \text{Factor} \\ &\mid \text{Term} / \text{Factor} \\ &\mid \text{Factor} \end{aligned}$$

Die umgewandelte Grammatik

$$\begin{aligned} \text{Expr} &\rightarrow \text{Term Expr}' \\ \text{Expr}' &\rightarrow + \text{Term Expr}' \\ &\mid - \text{Term Expr}' \\ &\mid \varepsilon \end{aligned}$$

$$\begin{aligned} \text{Links} &\rightarrow \beta_1 \text{Rechts} \mid \dots \mid \beta_n \text{Rechts} \\ \text{Rechts} &\rightarrow \alpha_1 \text{Rechts} \mid \dots \mid \alpha_m \text{Rechts} \mid \varepsilon \end{aligned}$$

$$\begin{aligned} \text{Term} &\rightarrow \text{Factor Term}' \\ \text{Term}' &\rightarrow * \text{Factor Term}' \\ &\mid / \text{Factor Term}' \\ &\mid \varepsilon \end{aligned}$$

Diese Grammatik ist nicht linksrekursiv und erhält die Präzedenz.

Fehlererkennung

- 4 Fehlerklassen:
 - **Lexikalische Fehler**, z.B. falsche Schreibung von Bezeichnern
 - **Syntaktische Fehler**, z.B. fehlende Semikola/Klammern
 - **Semantische Fehler**, z.B. Nichtübereinstimmung von Typen
 - **Logische Fehler**, z.B. Fehler trotz wohlgeformtem Quellcode
- Ein Parser sollte auf jeden Fall **Syntaxfehler** in der Eingabe **erkennen**.
- Manche Parser können Fehler **beheben** bzw. Änderungsvorschläge machen.
- Manche Parser können einen Fehler **intern** so **korrigieren**, dass sie das Parsen **fortsetzen** und noch weitere Fehler erkennen können.

Syntaxfehler

Beispiel: $a=a*(b+c*d; b=5-;$

Gültiger Präfix

$a=a*(b+c*+$

kein gültiger Präfix

Gültiger Präfix

- Ein Wort u ist ein gültiger Präfix, wenn es ein Terminalwort w gibt, so dass uw ein gültiges Programm ist.

Die in der Vorlesung betrachteten Parsingmethoden (LL und LR) erkennen gültige Präfixe.

Parsing-Techniken

Top-down Parser *(predictive (LL(1)), recursive descent, ...)*

- Beginne an der Wurzel des Syntaxbaums und entwickle ihn in Richtung der Blätter.
- Wähle Produktionsregel und versuche damit die Eingabe abzudecken.
- Falscher “Zug” $\Rightarrow$ mache Rückzieher (Backtracking)
- Es gibt auch “backtrack-freie” Grammatiken.

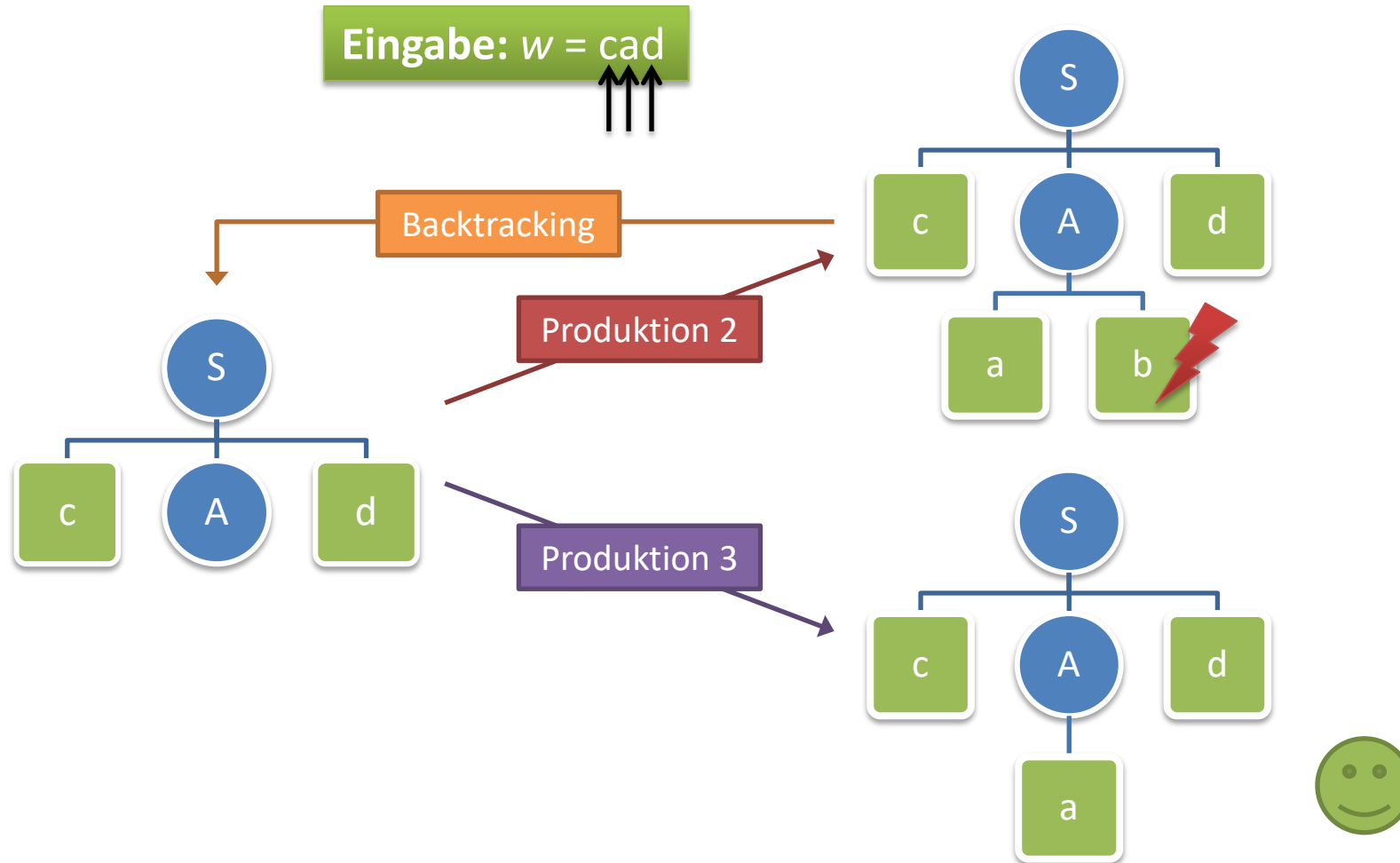
Bottom-up Parser *(shift-reduce (LR(1)), operator precedence, ...)*

- Starte an den Blättern und entwickle den Baum in Richtung der Wurzel.
- Wenn Zeichen von der Eingabe gelesen werden, wird sich im Zustand gemerkt, welche Möglichkeiten (Rümpfe einer Produktion) noch bestehen.
- Starte in einem Zustand, der für alle möglichen ersten Tokens gültig ist.
 - Eingaben mit ungültigen ersten Tokens werden sofort verworfen

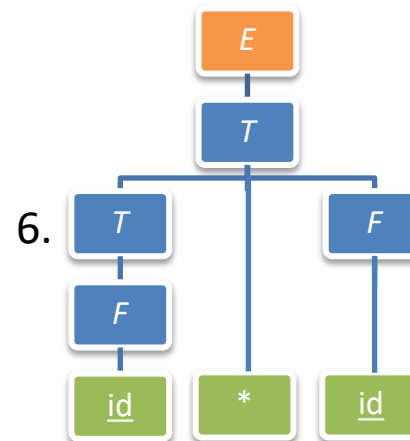
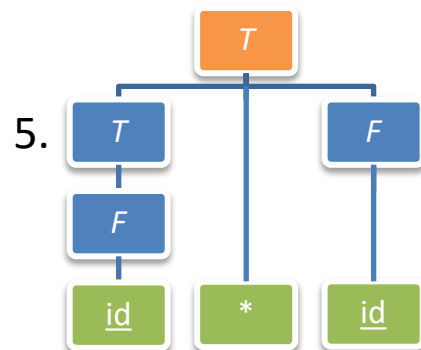
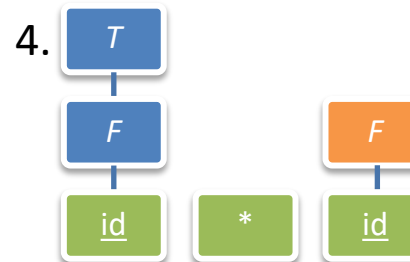
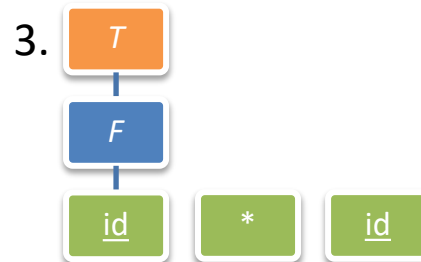
Backtracking

Grammatik

1	$S \rightarrow c A d$
2	$A \rightarrow a b$
3	$\quad \mid a$



Bottom-Up Parsing



Grammatik

1	$Expr \rightarrow Expr + Term$
2	$\quad \quad \quad \quad Term$
3	$Term \rightarrow Term * Factor$
4	$\quad \quad \quad \quad Factor$
5	$Factor \rightarrow (Expr)$
6	$\quad \quad \quad \quad id$

Parsen mit Vorausschau (Predictive Parsing)

- Parser können “vorhersagen”, welche Produktionsregel verwendet werden muss.
 - durch Anschauen einiger der nächsten Tokens
 - kein Backtracking
- Vorausschauende Parser existieren für bestimmte Klassen von Grammatiken, z.B. LL(k) und LR(k)
 - L bedeutet “links-nach-rechts” Verarbeitung der Eingabe
 - L bedeutet “Linksableitung”
 - R bedeutet “Rückwärts-Rechtsableitung”
 - k bedeutet “Vorausschau beruht auf k Tokens”



Kontextfreie Grammatiken

LR(1)

LL(1)

RG